# SIMATS ENGINEERING
## ASSESSMENT TOOL 1

**COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN**
**COURSE CODE: CSA1104**

---

**COURSE OUTCOME COVERED**

***CO1:*** *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems.* *(BL2, BL3)*

*Assessment Tool Weightage: 60%*

---

**QUESTIONS: 12** **Total Marks:60**

Annexure A

**CONCEPT MAPPING**

---

***Code of Conduct:***

*I _Dharshini/192521408_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

*Dharshini*

Annexure A

**CONCEPT MAPPING**

# 1. Construct a Concept Map

**Given:**

Requirement Analysis → _____ → Coding → Testing → _____ → Maintenance

**Answer:**

The missing concepts are:

- Design
- Deployment

**Explanation:**

- Design converts requirements into system architecture.
- Deployment releases the software to end users.

**Analysis:**

- Ensures systematic software development.
- Reduces project risks.
- Improves software quality and maintainability.

**(5 Marks)**

---

# 2. Construct a Concept Map

**Given:**

Object-Oriented Principles

→ Abstraction

→ _____

→ Inheritance

→ _____

**Answer:**

The missing concepts are:

- Encapsulation
- Polymorphism

**Explanation:**

- Encapsulation protects data from unauthorized access.
- Polymorphism allows one interface to support multiple behaviors.

**Analysis:**

- Improves security and flexibility.
- Enhances code reusability.
- Supports efficient software design.

**(5 Marks)**

---

# 3. Construct a Concept Map

**Given:**

Class → Blueprint

Object → Instance

Method → _____

Attribute → _____

**Answer:**

The missing concepts are:

- Behavior
- State

**Explanation:**

- Methods define the behavior of objects.
- Attributes represent the state of objects.

**Analysis:**

- Helps model real-world entities.
- Defines object characteristics and actions.
- Improves system organization.

**(5 Marks)**

---

# 4. A Concept Map Shows

**Given:**

Software Quality

→ Reliability

→ _____

→ Maintainability

**Answer:**

The missing concept is:

- Usability

**Explanation:**

- Usability measures how easily users interact with software.
- Reliable software performs consistently.
- Maintainable software is easy to update.

**Analysis:**

- Improves user satisfaction.
- Enhances product acceptance.
- Contributes to software success.

**(5 Marks)**

---

# 5. UML Concept Map

**Given:**

UML

→ Structural Diagrams → _____

→ Behavioral Diagrams → Activity Diagram

→ Interaction Diagrams → _____

**Answer:**

The missing concepts are:

- Object Diagram
- Communication Diagram

**Explanation:**

- Object diagrams represent object instances and relationships.
- Communication diagrams show message exchange among objects.

**Analysis:**

- Improves system visualization.
- Supports object-oriented analysis.
- Facilitates stakeholder communication.

**(5 Marks)**

---

# 6. Given:

Requirements ✔ → Design ✔ → Coding ✖ → Testing ✖ → Deployment ✖

**Explanation:**

- Failure occurs during the Coding phase.
- Program logic contains implementation errors.

**Analysis:**

- Errors affect subsequent phases.
- Testing cannot proceed effectively.
- Software quality decreases.

**Solution:**

- Review source code.
- Apply coding standards.
- Conduct code inspections before testing.

**(5 Marks)**

# 7. Construct a Concept Map

**Given:**

Problem Identification → _____ → Design → Implementation → _____

**Answer:**

The missing concepts are:

- Requirement Analysis
- Testing

**Explanation:**

- Requirement Analysis identifies user needs and system requirements.
- Testing verifies whether the implemented system meets the specified requirements.

**Analysis:**

- Improves software quality.
- Reduces development errors.
- Ensures successful project completion.

**(5 Marks)**

---

# 8. Construct a Concept Map

**Given:**

Object-Oriented Features

→ Class

→ _____

→ Message Passing

→ _____

**Answer:**

The missing concepts are:

- Object
- Dynamic Binding

**Explanation:**

- Objects are instances of classes.
- Dynamic binding allows method calls to be resolved at runtime.

**Analysis:**

- Supports flexibility in software design.

- Improves code maintainability.
- Enables runtime polymorphism.

**(5 Marks)**

---

# 9. Construct a Concept Map

**Given:**

Software Development Models

→ Waterfall

→ _____

→ Spiral

→ _____

**Answer:**

The missing concepts are:

- Incremental Model
- Agile Model

**Explanation:**

- Incremental development delivers software in stages.
- Agile supports iterative development with continuous customer feedback.

**Analysis:**

- Enhances adaptability.
- Reduces project risks.
- Improves customer satisfaction.

**(5 Marks)**

---

# 10. Construct a Concept Map

**Given:**

Class Diagram

→ Class

→ _____

→ Association

→ _____

**Answer:**

The missing concepts are:

- Attribute
- Method

**Explanation:**

- Attributes define the properties of a class.
- Methods define the operations performed by a class.

**Analysis:**

- Represents object structure effectively.
- Improves software documentation.
- Supports object-oriented design.

**(5 Marks)**

---

# 11. Construct a Concept Map

**Given:**

Software Engineering Principles

→ Modularity

→ _____

→ Reusability

→ _____

**Answer:**

The missing concepts are:

- Maintainability
- Scalability

**Explanation:**

- Maintainability simplifies software updates.
- Scalability enables systems to handle increased workload efficiently.

**Analysis:**

- Improves software performance.
- Reduces maintenance effort.
- Supports long-term software evolution.

**(5 Marks)**

## 12. Given:

Requirement Analysis ✔ → System Design ✔ → Coding ✔ → Integration ✖ → Deployment ✖

**Explanation:**

- Failure occurs during the Integration phase.
- Individual modules fail to communicate correctly.

**Analysis:**

- Causes functional inconsistencies.
- Delays software deployment.
- Increases debugging effort.

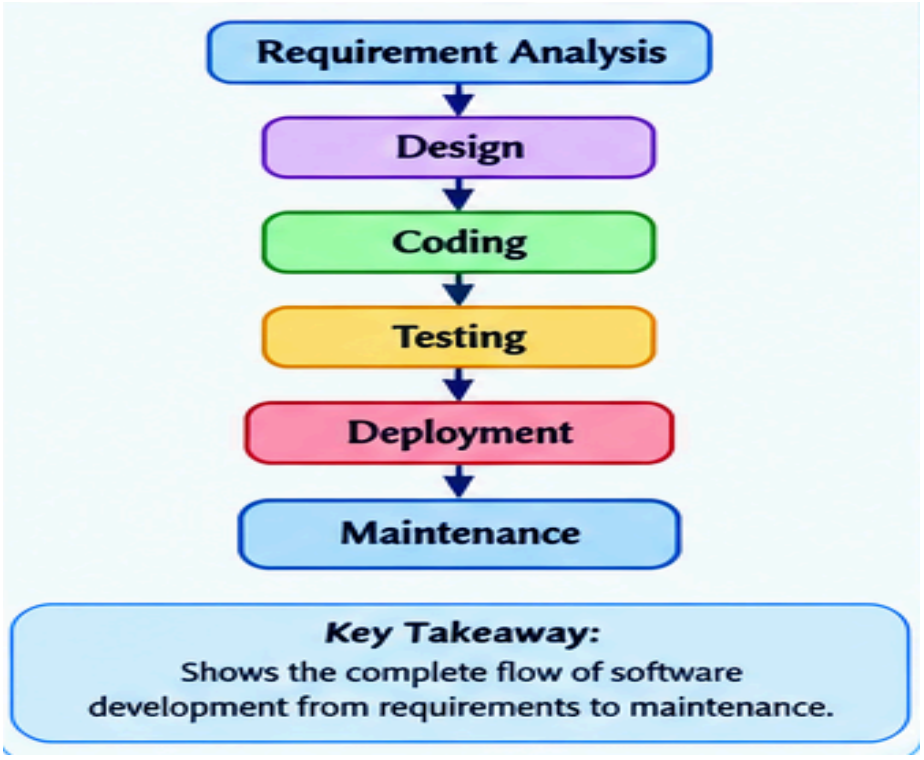
**Solution:**

- Perform integration testing.
- Verify module interfaces.
- Resolve compatibility issues before deployment.

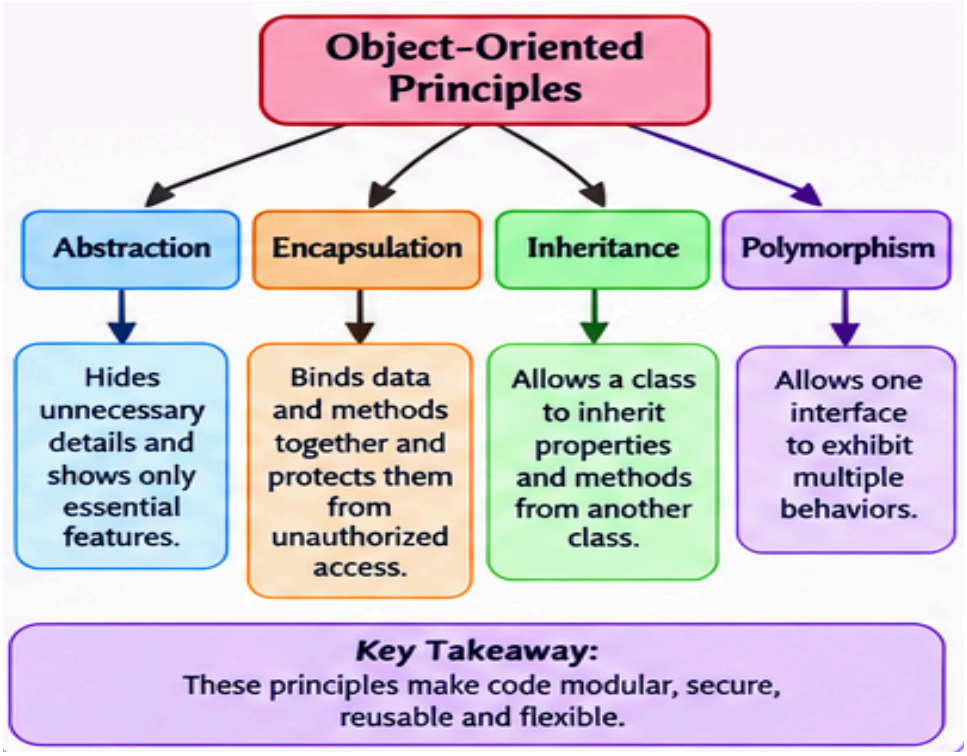
**(5 Marks)**

**Answers:**

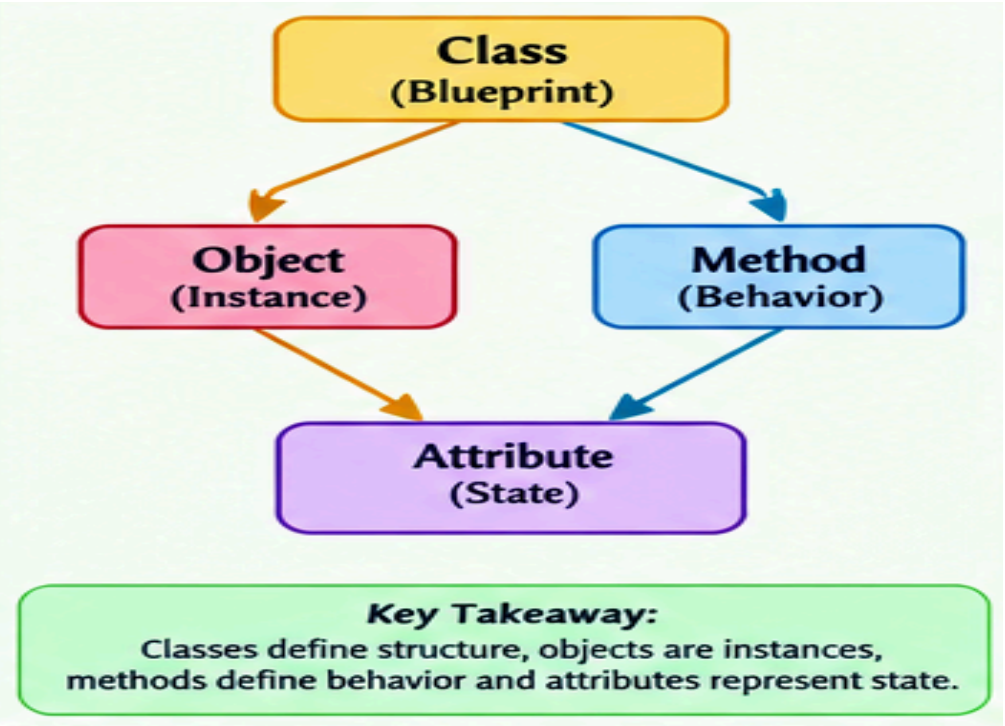
**1.**



2.


Object-Oriented Principles
Abstraction
Encapsulation
Inheritance
Polymorphism
Hides unnecessary details and shows only essential features.
Binds data and methods together and protects them from unauthorized access.
Allows a class to inherit properties and methods from another class.
Allows one interface to exhibit multiple behaviors.
Key Takeaway:
These principles make code modular, secure, reusable and flexible.

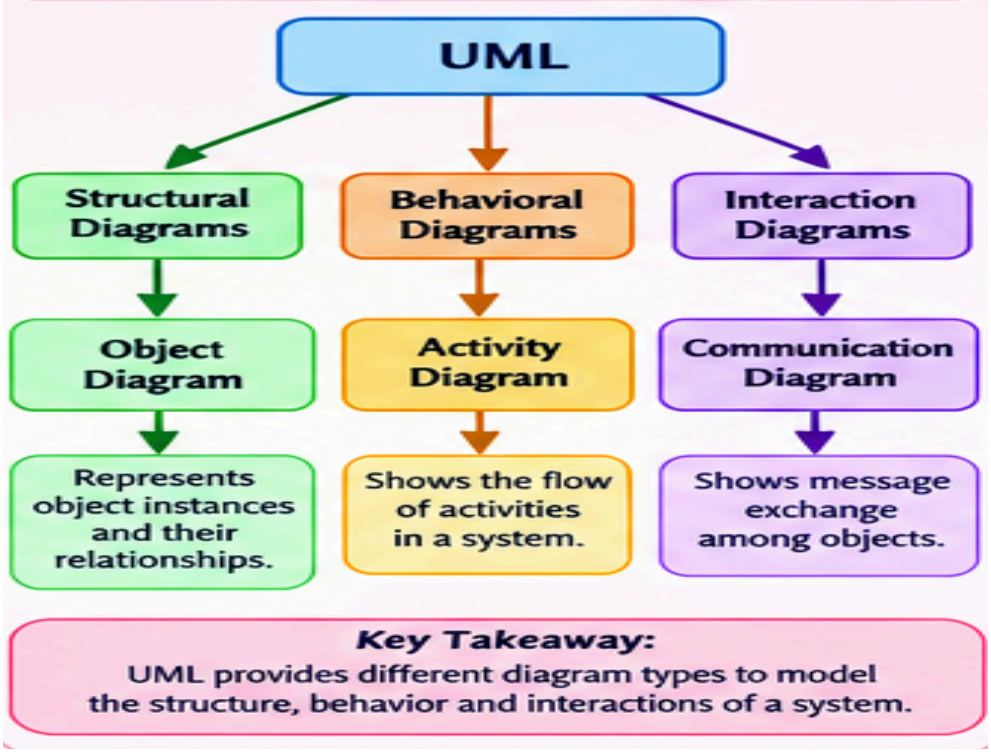
3.


Class
(Blueprint)
Object
(Instance)
Method
(Behavior)
Attribute
(State)
Key Takeaway:
Classes define structure, objects are instances, methods define behavior and attributes represent state.

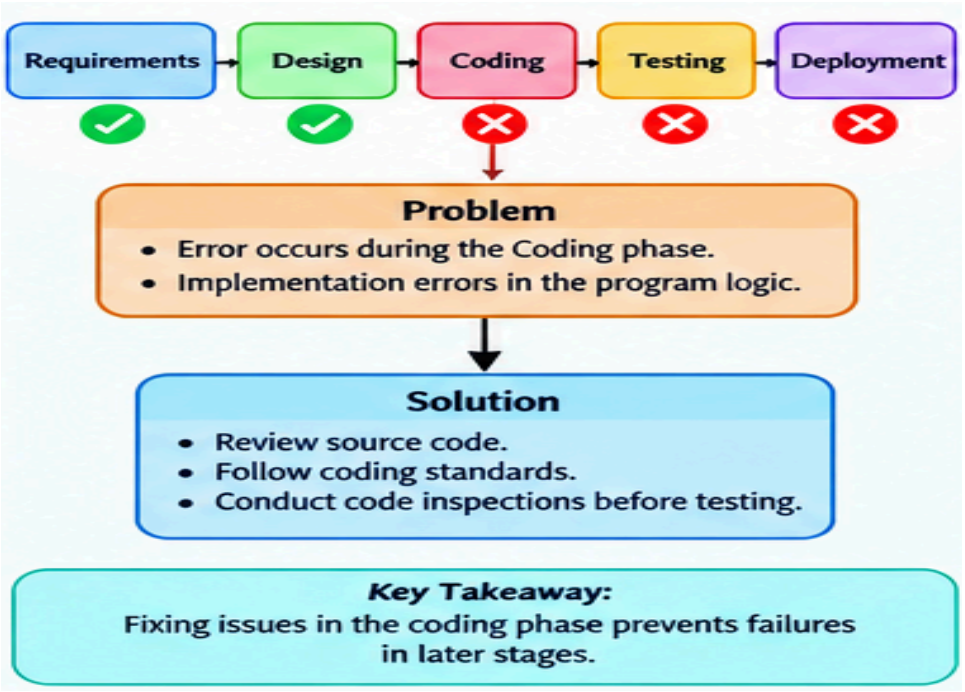
4.


Software Quality
Reliability
Usability
Maintainability
Performs consistently and correctly.
Easy for users to learn and interact with.
Easy to update and modify in the future.
Key Takeaway:
Quality software must be reliable, usable and maintainable for long-term success.

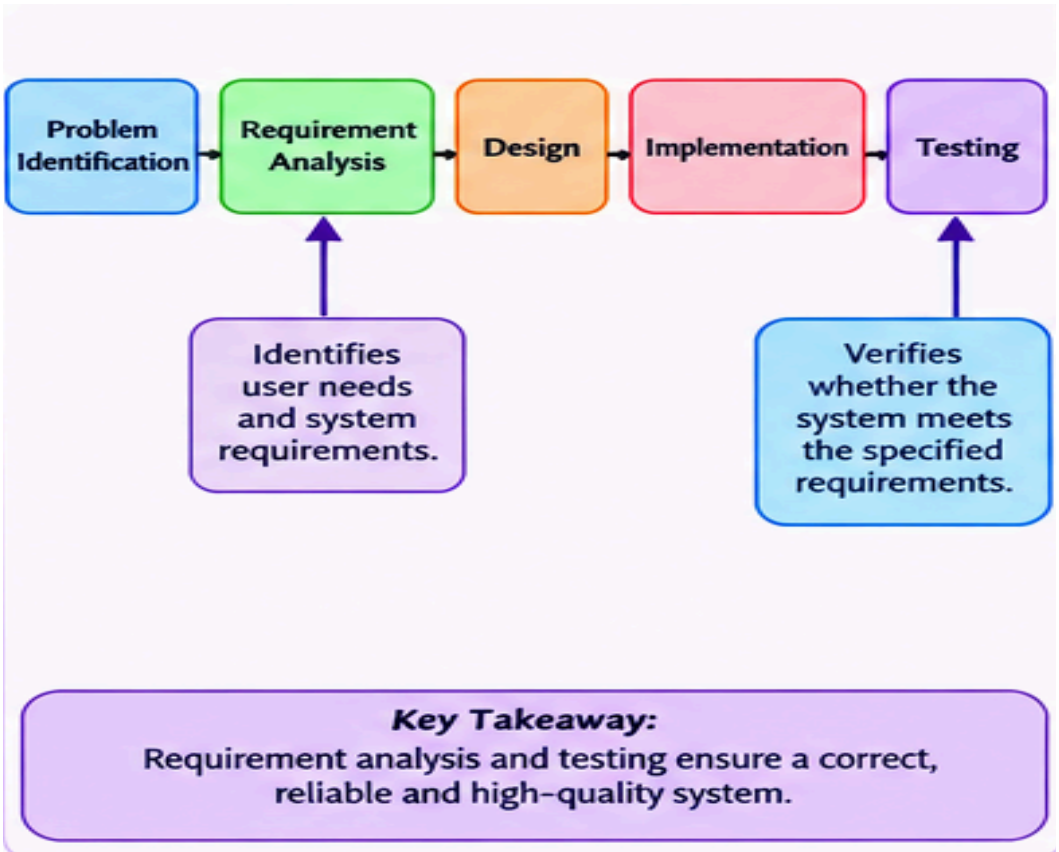
5.


UML
Structural Diagrams
Behavioral Diagrams
Interaction Diagrams
Object Diagram
Activity Diagram
Communication Diagram
Represents object instances and their relationships.
Shows the flow of activities in a system.
Shows message exchange among objects.
Key Takeaway:
UML provides different diagram types to model the structure, behavior and interactions of a system.

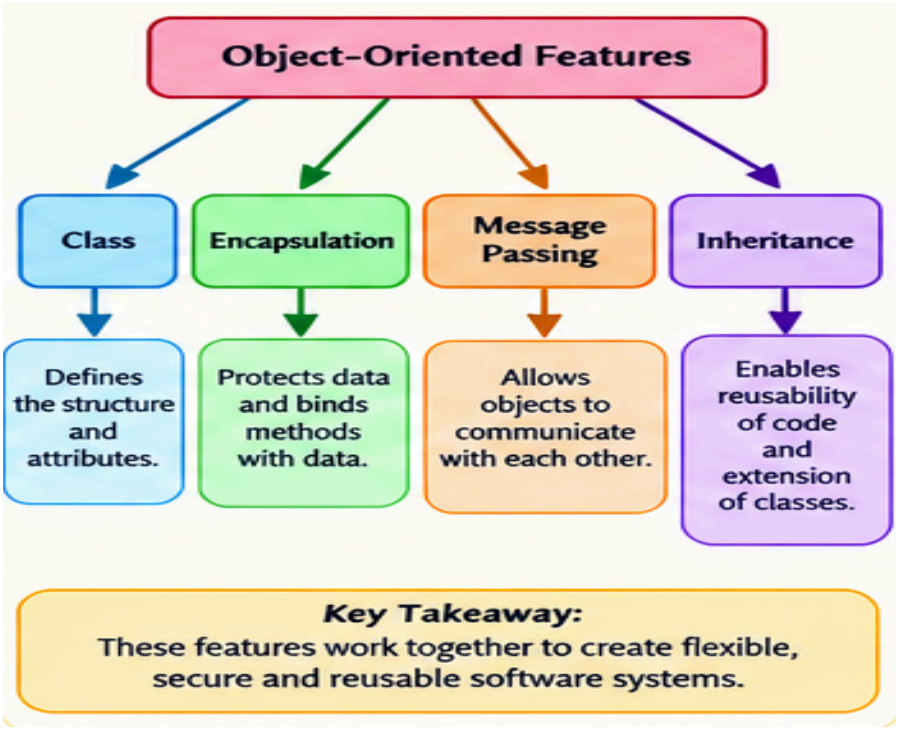
6.


Requirements
Design
Coding
Testing
Deployment
Problem
Error occurs during the Coding phase.
Implementation errors in the program logic.
Solution
Review source code.
Follow coding standards.
Conduct code inspections before testing.
Key Takeaway:
Fixing issues in the coding phase prevents failures in later stages.

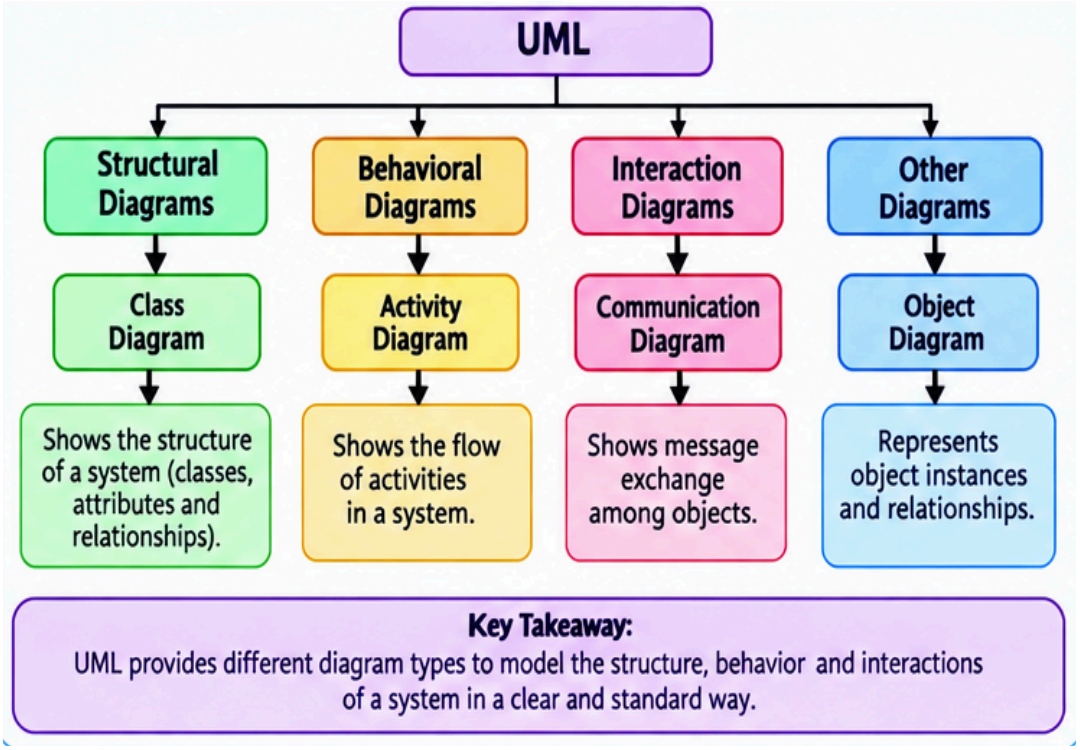
7.


Problem Identification
Requirement Analysis
Design
Implementation
Testing
Identifies user needs and system requirements.
Verifies whether the system meets the specified requirements.
Key Takeaway:
Requirement analysis and testing ensure a correct, reliable and high-quality system.

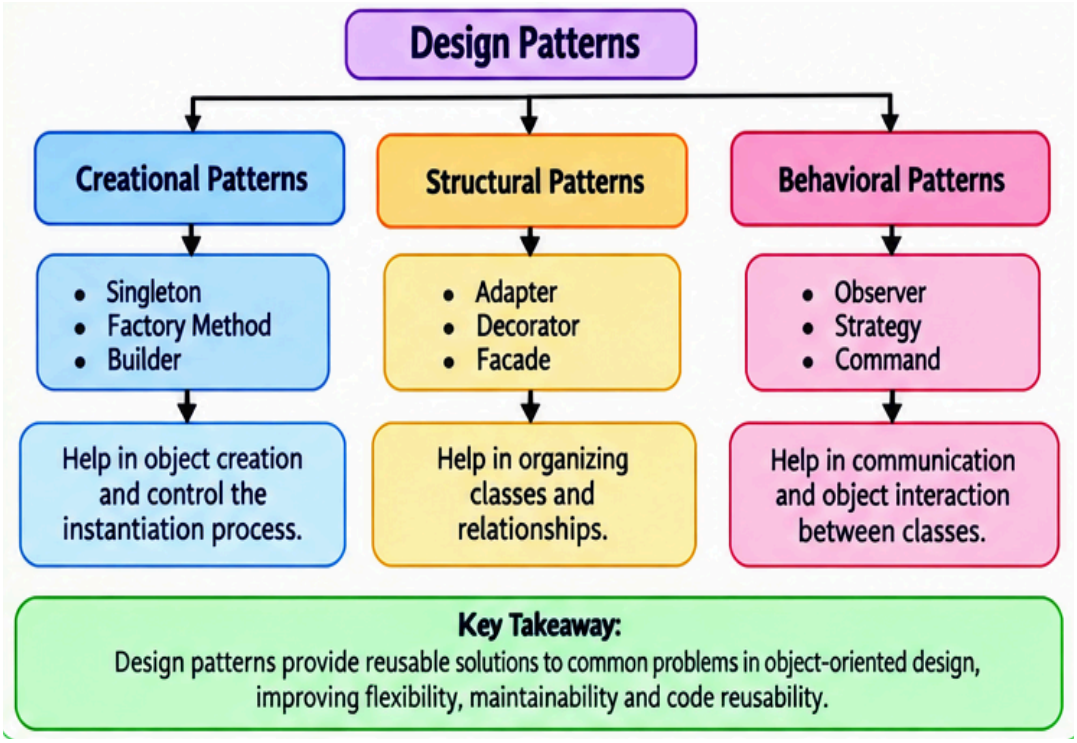
8.


Object-Oriented Features
Class
Encapsulation
Message Passing
Inheritance
Defines the structure and attributes.
Protects data and binds methods with data.
Allows objects to communicate with each other.
Enables reusability of code and extension of classes.
Key Takeaway:
These features work together to create flexible, secure and reusable software systems.

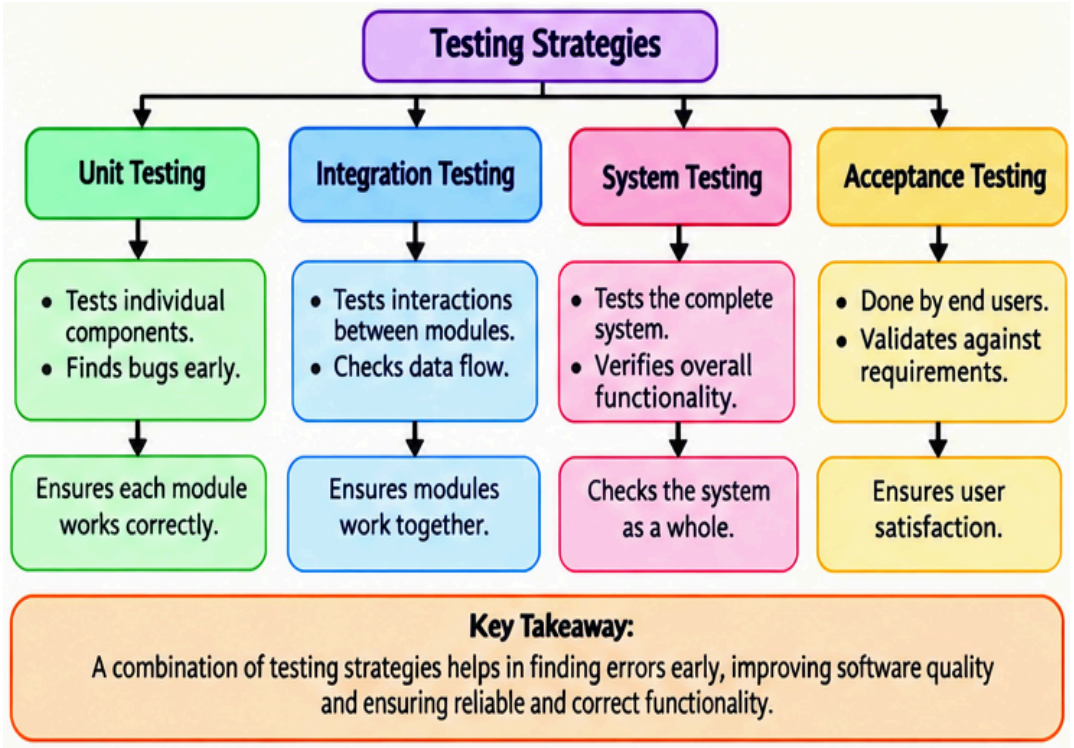
9.


UML
Structural Diagrams
Behavioral Diagrams
Interaction Diagrams
Other Diagrams
Class Diagram
Activity Diagram
Communication Diagram
Object Diagram
Shows the structure of a system (classes, attributes and relationships).
Shows the flow of activities in a system.
Shows message exchange among objects.
Represents object instances and relationships.
Key Takeaway:
UML provides different diagram types to model the structure, behavior and interactions of a system in a clear and standard way.

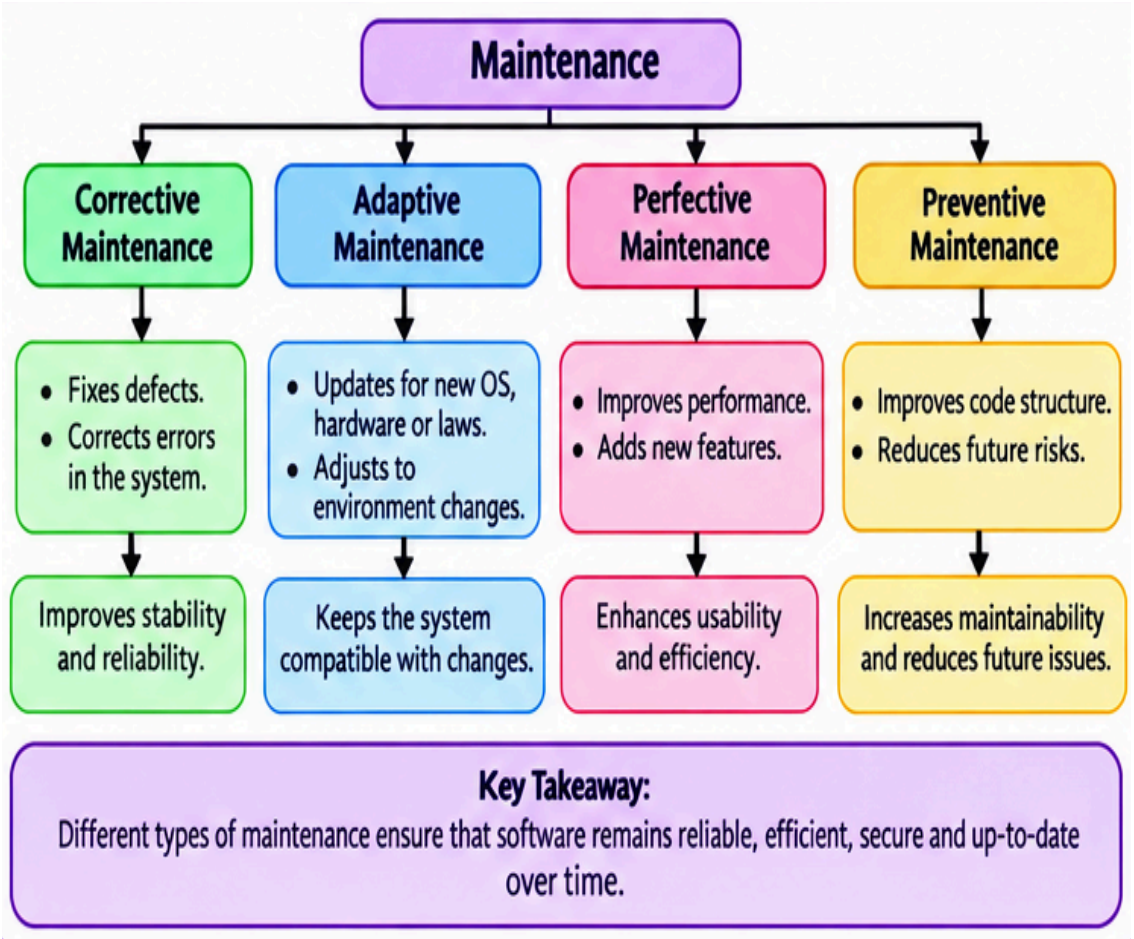
**10.**


Design Patterns
Creational Patterns
Structural Patterns
Behavioral Patterns
Singleton
Factory Method
Builder
Adapter
Decorator
Facade
Observer
Strategy
Command
Help in object creation and control the instantiation process.
Help in organizing classes and relationships.
Help in communication and object interaction between classes.
Key Takeaway:
Design patterns provide reusable solutions to common problems in object-oriented design, improving flexibility, maintainability and code reusability.

**11.**


Testing Strategies
Unit Testing
Integration Testing
System Testing
Acceptance Testing
Tests individual components.
Finds bugs early.
Tests interactions between modules.
Checks data flow.
Tests the complete system.
Verifies overall functionality.
Done by end users.
Validates against requirements.
Ensures each module works correctly.
Ensures modules work together.
Checks the system as a whole.
Ensures user satisfaction.
Key Takeaway:
A combination of testing strategies helps in finding errors early, improving software quality and ensuring reliable and correct functionality.

**12.**


Maintenance
Corrective Maintenance
Adaptive Maintenance
Perfective Maintenance
Preventive Maintenance
• Fixes defects.
• Corrects errors in the system.
• Updates for new OS, hardware or laws.
• Adjusts to environment changes.
• Improves performance.
• Adds new features.
• Improves code structure.
• Reduces future risks.
Improves stability and reliability.
Keeps the system compatible with changes.
Enhances usability and efficiency.
Increases maintainability and reduces future issues.
Key Takeaway:
Different types of maintenance ensure that software remains reliable, efficient, secure and up-to-date over time.

# CONCEPT MAPPING

## RUBRICS FOR CALCULATION

### CONCEPT MAPPING

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Concept Identification | 25% | Identifies all key concepts from literature accurately and comprehensively | Identifies most key concepts with minor omissions | Identifies limited concepts; some are irrelevant or missing | Fails to identify essential concepts |
| Linkages | 25% | Establishes clear, meaningful, and accurate relationships between concepts | Shows correct relationships with minor gaps | Relationships are weak, unclear, or partially incorrect | No meaningful relationships established |
| Organization | 20% | Concepts are well-structured hierarchically with logical flow and grouping | Mostly organized with minor structural issues | Some organization but lacks clear hierarchy | No logical organization or structure |
| Integration of Literature | 20% | Integrates multiple studies effectively to show connections and comparisons | Shows some integration of literature | Limited integration; mostly isolated concepts | No integration of literature evidence |
| Clarity | 10% | Concept map is clear, neat, visually structured, and easy to interpret | Generally clear with minor readability issues | Clarity is reduced; difficult to interpret in parts | Poorly presented and hard to understand |